

CF0

```
public static final int CF0
```

maps absolute notes to MIDI numbers

BSN1

```
public static final int BSN1
```

maps absolute notes to MIDI numbers

BN1

```
public static final int BN1
```

maps absolute notes to MIDI numbers

BFN1

```
public static final int BFN1
```

maps absolute notes to MIDI numbers

ASN1

```
public static final int ASN1
```

maps absolute notes to MIDI numbers

AN1

```
public static final int AN1
```

maps absolute notes to MIDI numbers

AFN1

```
public static final int AFN1
```

maps absolute notes to MIDI numbers

GSN1

```
public static final int GSN1
```

maps absolute notes to MIDI numbers

maps absolute notes to MIDI numbers

E0

```
public static final int E0
```

maps absolute notes to MIDI numbers

EF0

```
public static final int EF0
```

maps absolute notes to MIDI numbers

DS0

```
public static final int DS0
```

maps absolute notes to MIDI numbers

D0

```
public static final int D0
```

maps absolute notes to MIDI numbers

DF0

```
public static final int DF0
```

maps absolute notes to MIDI numbers

CS0

```
public static final int CS0
```

maps absolute notes to MIDI numbers

C0

```
public static final int C0
```

maps absolute notes to MIDI numbers

GN1

```
public static final int GN1
```

maps absolute notes to MIDI numbers

GFN1

```
public static final int GFN1
```

maps absolute notes to MIDI numbers

FSN1

```
public static final int FSN1
```

maps absolute notes to MIDI numbers

FN1

```
public static final int FN1
```

maps absolute notes to MIDI numbers

FFN1

```
public static final int FFN1
```

maps absolute notes to MIDI numbers

ESN1

```
public static final int ESN1
```

maps absolute notes to MIDI numbers

EN1

```
public static final int EN1
```

maps absolute notes to MIDI numbers

EFN1

```
public static final int EFN1
```


maps absolute notes to MIDI numbers

DSN1

```
public static final int DSN1
```

maps absolute notes to MIDI numbers

DN1

```
public static final int DN1
```

maps absolute notes to MIDI numbers

DFN1

```
public static final int DFN1
```

maps absolute notes to MIDI numbers

CSN1

```
public static final int CSN1
```

maps absolute notes to MIDI numbers

CN1

```
public static final int CN1
```

maps absolute notes to MIDI numbers

g9

```
public static final int g9
```

maps absolute notes to MIDI numbers

gf9

```
public static final int gf9
```

maps absolute notes to MIDI numbers

fs9

```
public static final int fs9
```

maps absolute notes to MIDI numbers

f9

```
public static final int f9
```

maps absolute notes to MIDI numbers

ff9

```
public static final int ff9
```

maps absolute notes to MIDI numbers

es9

```
public static final int es9
```

maps absolute notes to MIDI numbers

e9

```
public static final int e9
```

maps absolute notes to MIDI numbers

ef9

```
public static final int ef9
```

maps absolute notes to MIDI numbers

ds9

```
public static final int ds9
```

maps absolute notes to MIDI numbers

d9

```
public static final int d9
```

maps absolute notes to MIDI numbers

df9

```
public static final int df9
```

maps absolute notes to MIDI numbers

cs9

```
public static final int cs9
```

maps absolute notes to MIDI numbers

c9

```
public static final int c9
```

maps absolute notes to MIDI numbers

cf9

```
public static final int cf9
```

maps absolute notes to MIDI numbers

bs8

```
public static final int bs8
```

maps absolute notes to MIDI numbers

b8

```
public static final int b8
```

maps absolute notes to MIDI numbers

bf8

```
public static final int bf8
```

maps absolute notes to MIDI numbers

as8

```
public static final int as8
```


maps absolute notes to MIDI numbers

a8

```
public static final int a8
```

maps absolute notes to MIDI numbers

af8

```
public static final int af8
```

maps absolute notes to MIDI numbers

gs8

```
public static final int gs8
```

maps absolute notes to MIDI numbers

g8

```
public static final int g8
```

maps absolute notes to MIDI numbers

gf8

```
public static final int gf8
```

maps absolute notes to MIDI numbers

fs8

```
public static final int fs8
```

maps absolute notes to MIDI numbers

f8

```
public static final int f8
```

maps absolute notes to MIDI numbers

ff8

```
public static final int ff8
```

maps absolute notes to MIDI numbers

es8

```
public static final int es8
```

maps absolute notes to MIDI numbers

e8

```
public static final int e8
```

maps absolute notes to MIDI numbers

ef8

```
public static final int ef8
```

maps absolute notes to MIDI numbers

ds8

```
public static final int ds8
```

maps absolute notes to MIDI numbers

d8

```
public static final int d8
```

maps absolute notes to MIDI numbers

df8

```
public static final int df8
```

maps absolute notes to MIDI numbers

cs8

```
public static final int cs8
```

maps absolute notes to MIDI numbers

c8

`public static final int c8`

maps absolute notes to MIDI numbers

cf8

`public static final int cf8`

maps absolute notes to MIDI numbers

bs7

`public static final int bs7`

maps absolute notes to MIDI numbers

b7

`public static final int b7`

maps absolute notes to MIDI numbers

bf7

`public static final int bf7`

maps absolute notes to MIDI numbers

as7

`public static final int as7`

maps absolute notes to MIDI numbers

a7

`public static final int a7`

maps absolute notes to MIDI numbers

af7

`public static final int af7`

maps absolute notes to MIDI numbers

gs7

`public static final int gs7`

maps absolute notes to MIDI numbers

g7

`public static final int g7`

maps absolute notes to MIDI numbers

gf7

`public static final int gf7`

maps absolute notes to MIDI numbers

fs7

`public static final int fs7`

maps absolute notes to MIDI numbers

f7

`public static final int f7`

maps absolute notes to MIDI numbers

ff7

`public static final int ff7`

maps absolute notes to MIDI numbers

es7

`public static final int es7`

maps absolute notes to MIDI numbers

e7

```
public static final int e7
```

maps absolute notes to MIDI numbers

ef7

```
public static final int ef7
```

maps absolute notes to MIDI numbers

ds7

```
public static final int ds7
```

maps absolute notes to MIDI numbers

d7

```
public static final int d7
```

maps absolute notes to MIDI numbers

df7

```
public static final int df7
```

maps absolute notes to MIDI numbers

cs7

```
public static final int cs7
```

maps absolute notes to MIDI numbers

c7

```
public static final int c7
```

maps absolute notes to MIDI numbers

cf7

```
public static final int cf7
```

maps absolute notes to MIDI numbers

bs6

```
public static final int bs6
```

maps absolute notes to MIDI numbers

b6

```
public static final int b6
```

maps absolute notes to MIDI numbers

bf6

```
public static final int bf6
```

maps absolute notes to MIDI numbers

as6

```
public static final int as6
```

maps absolute notes to MIDI numbers

a6

```
public static final int a6
```

maps absolute notes to MIDI numbers

af6

```
public static final int af6
```

maps absolute notes to MIDI numbers

gs6

```
public static final int gs6
```

maps absolute notes to MIDI numbers

g6

```
public static final int g6
```

maps absolute notes to MIDI numbers

gf6

```
public static final int gf6
```

maps absolute notes to MIDI numbers

fs6

```
public static final int fs6
```

maps absolute notes to MIDI numbers

f6

```
public static final int f6
```

maps absolute notes to MIDI numbers

ff6

```
public static final int ff6
```

maps absolute notes to MIDI numbers

es6

```
public static final int es6
```

maps absolute notes to MIDI numbers

e6

```
public static final int e6
```

maps absolute notes to MIDI numbers

ef6

```
public static final int ef6
```

maps absolute notes to MIDI numbers

ds6

```
public static final int ds6
```

maps absolute notes to MIDI numbers

d6

```
public static final int d6
```

maps absolute notes to MIDI numbers

df6

```
public static final int df6
```

maps absolute notes to MIDI numbers

cs6

```
public static final int cs6
```

maps absolute notes to MIDI numbers

c6

```
public static final int c6
```

maps absolute notes to MIDI numbers

cf6

```
public static final int cf6
```

maps absolute notes to MIDI numbers

bs5

```
public static final int bs5
```

maps absolute notes to MIDI numbers

b5

```
public static final int b5
```

maps absolute notes to MIDI numbers

bf5

```
public static final int bf5
```

maps absolute notes to MIDI numbers

as5

```
public static final int as5
```

maps absolute notes to MIDI numbers

a5

```
public static final int a5
```

maps absolute notes to MIDI numbers

af5

```
public static final int af5
```

maps absolute notes to MIDI numbers

gs5

```
public static final int gs5
```

maps absolute notes to MIDI numbers

g5

```
public static final int g5
```

maps absolute notes to MIDI numbers

gf5

```
public static final int gf5
```

maps absolute notes to MIDI numbers

fs5

```
public static final int fs5
```


maps absolute notes to MIDI numbers

f5

```
public static final int f5
```

maps absolute notes to MIDI numbers

ff5

```
public static final int ff5
```

maps absolute notes to MIDI numbers

es5

```
public static final int es5
```

maps absolute notes to MIDI numbers

e5

```
public static final int e5
```

maps absolute notes to MIDI numbers

ef5

```
public static final int ef5
```

maps absolute notes to MIDI numbers

ds5

```
public static final int ds5
```

maps absolute notes to MIDI numbers

d5

```
public static final int d5
```

maps absolute notes to MIDI numbers

df5

```
public static final int df5
```

maps absolute notes to MIDI numbers

cs5

```
public static final int cs5
```

maps absolute notes to MIDI numbers

c5

```
public static final int c5
```

maps absolute notes to MIDI numbers

cf5

```
public static final int cf5
```

maps absolute notes to MIDI numbers

bs4

```
public static final int bs4
```

maps absolute notes to MIDI numbers

b4

```
public static final int b4
```

maps absolute notes to MIDI numbers

bf4

```
public static final int bf4
```

maps absolute notes to MIDI numbers

as4

```
public static final int as4
```

maps absolute notes to MIDI numbers

a4

`public static final int a4`

maps absolute notes to MIDI numbers

af4

`public static final int af4`

maps absolute notes to MIDI numbers

gs4

`public static final int gs4`

maps absolute notes to MIDI numbers

g4

`public static final int g4`

maps absolute notes to MIDI numbers

gf4

`public static final int gf4`

maps absolute notes to MIDI numbers

fs4

`public static final int fs4`

maps absolute notes to MIDI numbers

f4

`public static final int f4`

maps absolute notes to MIDI numbers

ff4

`public static final int ff4`

maps absolute notes to MIDI numbers

es4

```
public static final int es4
```

maps absolute notes to MIDI numbers

e4

```
public static final int e4
```

maps absolute notes to MIDI numbers

ef4

```
public static final int ef4
```

maps absolute notes to MIDI numbers

ds4

```
public static final int ds4
```

maps absolute notes to MIDI numbers

d4

```
public static final int d4
```

maps absolute notes to MIDI numbers

df4

```
public static final int df4
```

maps absolute notes to MIDI numbers

cs4

```
public static final int cs4
```

maps absolute notes to MIDI numbers

c4

```
public static final int c4
```

maps absolute notes to MIDI numbers

cf4

```
public static final int cf4
```

maps absolute notes to MIDI numbers

bs3

```
public static final int bs3
```

maps absolute notes to MIDI numbers

b3

```
public static final int b3
```

maps absolute notes to MIDI numbers

bf3

```
public static final int bf3
```

maps absolute notes to MIDI numbers

as3

```
public static final int as3
```

maps absolute notes to MIDI numbers

a3

```
public static final int a3
```

maps absolute notes to MIDI numbers

af3

```
public static final int af3
```

maps absolute notes to MIDI numbers

gs3

```
public static final int gs3
```

maps absolute notes to MIDI numbers

g3

```
public static final int g3
```

maps absolute notes to MIDI numbers

gf3

```
public static final int gf3
```

maps absolute notes to MIDI numbers

fs3

```
public static final int fs3
```

maps absolute notes to MIDI numbers

f3

```
public static final int f3
```

maps absolute notes to MIDI numbers

ff3

```
public static final int ff3
```

maps absolute notes to MIDI numbers

es3

```
public static final int es3
```

maps absolute notes to MIDI numbers

e3

```
public static final int e3
```

maps absolute notes to MIDI numbers

ef3

```
public static final int ef3
```

maps absolute notes to MIDI numbers

ds3

```
public static final int ds3
```

maps absolute notes to MIDI numbers

d3

```
public static final int d3
```

maps absolute notes to MIDI numbers

df3

```
public static final int df3
```

maps absolute notes to MIDI numbers

cs3

```
public static final int cs3
```

maps absolute notes to MIDI numbers

c3

```
public static final int c3
```

maps absolute notes to MIDI numbers

cf3

```
public static final int cf3
```

maps absolute notes to MIDI numbers

bs2

```
public static final int bs2
```

maps absolute notes to MIDI numbers

b2

```
public static final int b2
```

maps absolute notes to MIDI numbers

bf2

```
public static final int bf2
```

maps absolute notes to MIDI numbers

as2

```
public static final int as2
```

maps absolute notes to MIDI numbers

a2

```
public static final int a2
```

maps absolute notes to MIDI numbers

af2

```
public static final int af2
```

maps absolute notes to MIDI numbers

gs2

```
public static final int gs2
```

maps absolute notes to MIDI numbers

g2

```
public static final int g2
```

maps absolute notes to MIDI numbers

gf2

`public static final int gf2`

maps absolute notes to MIDI numbers

fs2

`public static final int fs2`

maps absolute notes to MIDI numbers

f2

`public static final int f2`

maps absolute notes to MIDI numbers

ff2

`public static final int ff2`

maps absolute notes to MIDI numbers

es2

`public static final int es2`

maps absolute notes to MIDI numbers

e2

`public static final int e2`

maps absolute notes to MIDI numbers

ef2

`public static final int ef2`

maps absolute notes to MIDI numbers

ds2

`public static final int ds2`

maps absolute notes to MIDI numbers

d2

```
public static final int d2
```

maps absolute notes to MIDI numbers

df2

```
public static final int df2
```

maps absolute notes to MIDI numbers

cs2

```
public static final int cs2
```

maps absolute notes to MIDI numbers

c2

```
public static final int c2
```

maps absolute notes to MIDI numbers

cf2

```
public static final int cf2
```

maps absolute notes to MIDI numbers

bs1

```
public static final int bs1
```

maps absolute notes to MIDI numbers

b1

```
public static final int b1
```

maps absolute notes to MIDI numbers

bf1

```
public static final int bf1
```

maps absolute notes to MIDI numbers

as1

```
public static final int as1
```

maps absolute notes to MIDI numbers

a1

```
public static final int a1
```

maps absolute notes to MIDI numbers

af1

```
public static final int af1
```

maps absolute notes to MIDI numbers

gs1

```
public static final int gs1
```

maps absolute notes to MIDI numbers

g1

```
public static final int g1
```

maps absolute notes to MIDI numbers

gf1

```
public static final int gf1
```

maps absolute notes to MIDI numbers

fs1

```
public static final int fs1
```

maps absolute notes to MIDI numbers

f1

`public static final int f1`

maps absolute notes to MIDI numbers

ff1

`public static final int ff1`

maps absolute notes to MIDI numbers

es1

`public static final int es1`

maps absolute notes to MIDI numbers

e1

`public static final int e1`

maps absolute notes to MIDI numbers

ef1

`public static final int ef1`

maps absolute notes to MIDI numbers

ds1

`public static final int ds1`

maps absolute notes to MIDI numbers

d1

`public static final int d1`

maps absolute notes to MIDI numbers

df1

`public static final int df1`

maps absolute notes to MIDI numbers

cs1

```
public static final int cs1
```

maps absolute notes to MIDI numbers

c1

```
public static final int c1
```

maps absolute notes to MIDI numbers

cf1

```
public static final int cf1
```

maps absolute notes to MIDI numbers

bs0

```
public static final int bs0
```

maps absolute notes to MIDI numbers

b0

```
public static final int b0
```

maps absolute notes to MIDI numbers

bf0

```
public static final int bf0
```

maps absolute notes to MIDI numbers

as0

```
public static final int as0
```

maps absolute notes to MIDI numbers

a0

```
public static final int a0
```

maps absolute notes to MIDI numbers

af0

```
public static final int af0
```

maps absolute notes to MIDI numbers

gs0

```
public static final int gs0
```

maps absolute notes to MIDI numbers

g0

```
public static final int g0
```

maps absolute notes to MIDI numbers

gf0

```
public static final int gf0
```

maps absolute notes to MIDI numbers

fs0

```
public static final int fs0
```

maps absolute notes to MIDI numbers

f0

```
public static final int f0
```

maps absolute notes to MIDI numbers

ff0

```
public static final int ff0
```

maps absolute notes to MIDI numbers

es0

```
public static final int es0
```

maps absolute notes to MIDI numbers

e0

```
public static final int e0
```

maps absolute notes to MIDI numbers

ef0

```
public static final int ef0
```

maps absolute notes to MIDI numbers

ds0

```
public static final int ds0
```

maps absolute notes to MIDI numbers

d0

```
public static final int d0
```

maps absolute notes to MIDI numbers

df0

```
public static final int df0
```

maps absolute notes to MIDI numbers

cs0

```
public static final int cs0
```

maps absolute notes to MIDI numbers

c0

```
public static final int c0
```


maps absolute notes to MIDI numbers

cf0

```
public static final int cf0
```

maps absolute notes to MIDI numbers

bsn1

```
public static final int bsn1
```

maps absolute notes to MIDI numbers

bn1

```
public static final int bn1
```

maps absolute notes to MIDI numbers

bf1

```
public static final int bf1
```

maps absolute notes to MIDI numbers

asn1

```
public static final int asn1
```

maps absolute notes to MIDI numbers

an1

```
public static final int an1
```

maps absolute notes to MIDI numbers

af1

```
public static final int af1
```

maps absolute notes to MIDI numbers

gsn1

```
public static final int gsn1
```

maps absolute notes to MIDI numbers

gn1

```
public static final int gn1
```

maps absolute notes to MIDI numbers

gfn1

```
public static final int gfn1
```

maps absolute notes to MIDI numbers

fsn1

```
public static final int fsn1
```

maps absolute notes to MIDI numbers

fn1

```
public static final int fn1
```

maps absolute notes to MIDI numbers

ffn1

```
public static final int ffn1
```

maps absolute notes to MIDI numbers

esn1

```
public static final int esn1
```

maps absolute notes to MIDI numbers

en1

```
public static final int en1
```

maps absolute notes to MIDI numbers

efn1

```
public static final int efn1
```

maps absolute notes to MIDI numbers

dsn1

```
public static final int dsn1
```

maps absolute notes to MIDI numbers

dn1

```
public static final int dn1
```

maps absolute notes to MIDI numbers

dfn1

```
public static final int dfn1
```

maps absolute notes to MIDI numbers

csn1

```
public static final int csn1
```

maps absolute notes to MIDI numbers

cn1

```
public static final int cn1
```

maps absolute notes to MIDI numbers

TOTALPITCHES

```
public static final int TOTALPITCHES
```

SEMITONES

```
public static final int SEMITONES
```

MIN_PITCH

```
public static final int MIN_PITCH
```

MAX_PITCH

```
public static final int MAX_PITCH
```

MIN_INTERVAL_SIZE

```
public static final int MIN_INTERVAL_SIZE
```

MAX_INTERVAL_SIZE

```
public static final int MAX_INTERVAL_SIZE
```

MIN_NOTE_DURATION

```
public static final int MIN_NOTE_DURATION
```

MAX_NOTE_DURATION

```
public static final int MAX_NOTE_DURATION
```

MODF

```
public static final int MODF
```

MODC

```
public static final int MODC
```

MODG

```
public static final int MODG
```

MODD

```
public static final int MODD
```

MODA

```
public static final int MODA
```

MODE

```
public static final int MODE
```

MODB

```
public static final int MODB
```

CBMAJOR

```
public static final int CBMAJOR
```

GBMAJOR

```
public static final int GBMAJOR
```

DBMAJOR

```
public static final int DBMAJOR
```

ABMAJOR

```
public static final int ABMAJOR
```

EBMAJOR

```
public static final int EBMAJOR
```

BBMAJOR

```
public static final int BBMAJOR
```

FMAJOR

```
public static final int FMAJOR
```

CMAJOR

```
public static final int CMAJOR
```

GMAJOR

```
public static final int GMAJOR
```

DMAJOR

```
public static final int DMAJOR
```

AMAJOR

```
public static final int AMAJOR
```

EMAJOR

```
public static final int EMAJOR
```

BMAJOR

```
public static final int BMAJOR
```

FSMAJOR

```
public static final int FSMAJOR
```

CSMAJOR

```
public static final int CSMAJOR
```

T_NOTE

```
public static final char T_NOTE
```

T_CHORD

```
public static final char T_CHORD
```

T_SCALE

```
public static final char T_SCALE
```

T_COLOR

```
public static final char T_COLOR
```

T_APPROACH

```
public static final char T_APPROACH
```

T_REST

```
public static final char T_REST
```

T_RANDOM

```
public static final char T_RANDOM
```

NONE

```
public static final String NONE
```

FIRST_SCALE

```
public static final String FIRST_SCALE
```

IGNORE

```
public static final int IGNORE
```

OVERWRITE

```
public static final int OVERWRITE
```

SAVE

```
public static final int SAVE
```

STYLE_EXTENSION

```
public static final String STYLE_EXTENSION
```

Enum Constants.Accidental

imp

```
java.lang.Object
└─ java.lang.Enum<Constants.Accidental>
    └─ imp.Constants.Accidental
```

All Implemented Interfaces:

Comparable<Constants.Accidental>, Serializable

Enclosing class:

Constants

```
public static enum Constants.Accidental
extends Enum<Constants.Accidental>
```

a flag so a Note knows whether it should be sharp, flat, or natural

Enum Constant Summary	Page
<u>FLAT</u>	153
<u>NATURAL</u>	153
<u>NOTHING</u>	154
<u>SHARP</u>	153

Method Summary	Page
static Constants.Accidental <u>valueOf</u> (String name)	154
static Constants.Accidental[] <u>values</u> ()	154

Enum Constant Detail

NATURAL

```
public static final Constants.Accidental NATURAL
```

SHARP

```
public static final Constants.Accidental SHARP
```

FLAT

```
public static final Constants.Accidental FLAT
```

NOTHING

```
public static final Constants.Accidental NOTHING
```

Method Detail

values

```
public static Constants.Accidental[] values()
```

valueOf

```
public static Constants.Accidental valueOf(String name)
```


Enum Constants.ExtractMode

imp

```
java.lang.Object
├─ java.lang.Enum<Constants.ExtractMode>
│   └─ imp.Constants.ExtractMode
```

All Implemented Interfaces:

Comparable<Constants.ExtractMode>, Serializable

Enclosing class:

Constants

```
public static enum Constants.ExtractMode
extends Enum<Constants.ExtractMode>
```

Modes of extracting stuff for transfer

Enum Constant Summary	Page
<u>BOTH</u>	156
<u>CELL</u>	155
<u>CHORDS</u>	156
<u>IDIOM</u>	155
<u>LICK</u>	155
<u>MELODY</u>	156
<u>QUOTE</u>	156

Method Summary	Page
static Constants.ExtractMode <u>valueOf</u> (String name)	156
static Constants.ExtractMode[] <u>values</u> ()	156

Enum Constant Detail

CELL

```
public static final Constants.ExtractMode CELL
```

IDIOM

```
public static final Constants.ExtractMode IDIOM
```

LICK

```
public static final Constants.ExtractMode LICK
```

QUOTE

```
public static final Constants.ExtractMode QUOTE
```

CHORDS

```
public static final Constants.ExtractMode CHORDS
```

MELODY

```
public static final Constants.ExtractMode MELODY
```

BOTH

```
public static final Constants.ExtractMode BOTH
```

Method Detail

values

```
public static Constants.ExtractMode[] values()
```

valueOf

```
public static Constants.ExtractMode valueOf(String name)
```

Enum Constants.StaveType

imp

```
java.lang.Object
├─ java.lang.Enum<Constants.StaveType>
│   └─ imp.Constants.StaveType
```

All Implemented Interfaces:

Comparable<Constants.StaveType>, Serializable

Enclosing class:

Constants

```
public static enum Constants.StaveType
extends Enum<Constants.StaveType>
```

enum indicating which stave the current window should display

Enum Constant Summary	Page
<u>AUTO</u>	158
<u>BASS</u>	157
<u>GRAND</u>	157
<u>NONE</u>	158
<u>TREBLE</u>	157

Method Summary	Page
static Constants.StaveType <u>valueOf</u> (String name)	158
static Constants.StaveType[] <u>values</u> ()	158

Enum Constant Detail

GRAND

```
public static final Constants.StaveType GRAND
```

TREBLE

```
public static final Constants.StaveType TREBLE
```

BASS

```
public static final Constants.StaveType BASS
```

AUTO

```
public static final Constants.StaveType AUTO
```

NONE

```
public static final Constants.StaveType NONE
```

Method Detail

values

```
public static Constants.StaveType[] values()
```

valueOf

```
public static Constants.StaveType valueOf(String name)
```


Package polya

Interface Summary		Page
<u>Function1</u>	Function1 is an interface defining the function argument to map	164
<u>Function2</u>	Function2 is an interface defining a 2-argument function	165

Class Summary		Page
<u>Arith</u>	Class Arth of plya package implements "polymorphic arithmetic"	160
<u>EOF</u>	EOF represents an end-of-file	162
<u>Growable</u>	Used as an argument to Seed	166
<u>Incremental</u>		167
<u>Polylist</u>	Polylist is the basic unit for constructing an open linked list of Objects.	170
<u>PolylistEnum</u>	PolylistEnum is an enumeration class for the class Polylist.	185
<u>Tokenizer</u>	Tokenizer provides a tokenizer of the package polya.	187

Exception Summary		Page
<u>eofException</u>	eofException is thrown when there is an end-of-file for Tokenizer	163

Class Arith

polya

```
java.lang.Object
└─ polya.Arith
```

```
public class Arith
extends Object
```

Class Arith of polya package implements "polymorphic arithmetic"

Author:

Robert Keller

Constructor Summary	Page
Arith()	160

Method Summary	Page
static Number add (Object v1, Object v2)	161
static Number divide (Object v1, Object v2)	161
static boolean equal (Object v1, Object v2)	161
static boolean greaterThan (Object v1, Object v2)	160
static boolean greaterThanOrEqual (Object v1, Object v2)	161
static Number mod (Object v1, Object v2)	161
static Number multiply (Object v1, Object v2)	161
static Number subtract (Object v1, Object v2)	161

Constructor Detail

Arith

```
public Arith()
```

Method Detail

greaterThan

```
public static boolean greaterThan(Object v1,
                                Object v2)
```


greaterThanOrEqualTo

```
public static boolean greaterThanOrEqualTo(Object v1,  
                                           Object v2)
```

equal

```
public static boolean equal(Object v1,  
                             Object v2)
```

add

```
public static Number add(Object v1,  
                           Object v2)
```

subtract

```
public static Number subtract(Object v1,  
                               Object v2)
```

multiply

```
public static Number multiply(Object v1,  
                               Object v2)
```

divide

```
public static Number divide(Object v1,  
                              Object v2)
```

mod

```
public static Number mod(Object v1,  
                           Object v2)
```


Class EOF

polya

```
java.lang.Object
└─ polya.EOF
```

```
public class EOF
extends Object
```

EOF represents an end-of-file

Author:

Robert Keller

Constructor Summary	
<u>EOF()</u>	162

Constructor Detail

EOF

```
public EOF()
```

Class eofException

polya

```
java.lang.Object
├── java.lang.Throwable
│   ├── java.lang.Exception
│   │   └── polya.eofException
```

All Implemented Interfaces:
Serializable

```
public class eofException
extends Exception
```

eofException is thrown when there is an end-of-file for Tokenizer

Author:
Robert Keller

Constructor Summary	Page
<u>eofException()</u>	163

Method Summary	Page
long <u>getSerialVersionUID()</u>	163

Constructor Detail

eofException

```
public eofException()
```

Method Detail

getSerialVersionUID

```
public long getSerialVersionUID()
```

Interface Function1

polya

```
public interface Function1
```

Function1 is an interface defining the function argument to map

Author:

Robert Keller

Method Summary		Page
Object	<u>apply</u> (Object x)	164

Method Detail

apply

Object **apply**(Object x)

Interface Function2

polya

```
public interface Function2
```

Function2 is an interface defining a 2-argument function

Author:

Robert Keller

Method Summary		Page
Object	<u>apply</u> (Object x, Object y)	165

Method Detail

apply

```
Object apply(Object x,  
             Object y)
```

Class Growable

polya

```
java.lang.Object
└─ polya.Growable
```

```
abstract public class Growable
extends Object
```

Used as an argument to Seed

Author:

Sean Hayes

See Also:

`polya.Seed`

Constructor Summary

Page

`Growable()`

166

Method Summary

Page

abstract
Object `grow()`

166

Constructor Detail

Growable

```
public Growable()
```

Method Detail

grow

```
public abstract Object grow()
```


Method Detail

first

```
public Object first()
```

first() returns the first element of a non-empty Polylist.

Overrides:

first in class Polylist

rest

```
public Polylist rest()
```

rest() returns the rest of a non-empty Polylist.

Overrides:

rest in class Polylist

isEmpty

```
public boolean isEmpty()
```

isEmpty() tells whether the Polylist is empty.

Overrides:

isEmpty in class Polylist

nonEmpty

```
public boolean nonEmpty()
```

nonEmpty() tells whether the Polylist is non-empty.

Overrides:

nonEmpty in class Polylist

toString

```
public String toString()
```

toString() converts Polylist to string, e.g. for printing

Overrides:

toString in class Polylist

ensureGrown

```
public void ensureGrown()
```

grown

```
public boolean grown()
```

getList

```
public Polylist getList()
```

Class Polylist

polya

```
java.lang.Object
└─ polya.Polylist
```

Direct Known Subclasses:

Incremental

```
public class Polylist
extends Object
```

Polylist is the basic unit for constructing an open linked list of Objects.
A Polylist can be either :
empty, or
non-empty, in which case it consists of:
a Object as the first thing in the list
a rest which is a Polylist.
A Polylist can also be a Incremental, which may be converted into an ordinary Polylist by applying methods such as isEmpty(), first(), and rest().

Author:

Robert Keller

Field Summary		Page
<u>static</u> <u>Polylist</u>	<u>nil</u> nil is the empty-list constant	173

Constructor Summary		Page
<u>Polylist</u> ()	construct empty Polylist	173
<u>Polylist</u> (Object First, <u>Polylist</u> Rest)	construct non-empty Polylist from a First and Rest	173
<u>Polylist</u> (Object First, <u>polya.Seed</u> Rest)	construct non-empty Polylist from a First and a Seed for the Rest	174

Method Summary		Page
<u>static</u> <u>String</u>	<u>analysis</u> (Object Ob) analysis produces a string analyzing objects, especially Polylists	183
<u>Polylist</u>	<u>append</u> (<u>Polylist</u> M) append(M) returns a Polylist consisting of the elements of this followed by those of M.	180
<u>Object[]</u>	<u>array</u> () array() returns an array of elements in list	183
<u>Polylist</u>	<u>assoc</u> (Object ob)	184
<u>static</u> <u>Polylist</u>	<u>assoc</u> (Object ob, <u>Polylist</u> alist)	184
<u>Polylist</u>	<u>cons</u> (Object First) cons returns a new Polylist given a First and this as a Rest	175

static Polylist	<u>cons</u> (Object First, Polylist Rest) static cons returns a new Polylist given a First and a Rest.	175
static Polylist	<u>cons</u> (Object First, polya.Seed Rest) This variant of cons takes a Seed instead of a Polylist as rest, so the list can be grown incrementally.	175
Polylist	<u>coprefix</u> (long n) coprefix creates the Polylist with all but the length-n prefix of a Polylist	182
PolylistEnum	<u>elements</u> () elements() returns a PolylistEnum object, which implements the interface java.util.Enumeration.	180
boolean	<u>equals</u> (Object M) equals(M) tells whether this Polylist is equal to some other Object	183
static boolean	<u>equals</u> (Polylist L, Polylist M) equals(L, M) tells whether Polylists L and M are equal	182
static Polylist	<u>explode</u> (String S) explode(String S) converts a string into a Polylist of Character	183
Object	<u>fifth</u> () fifth selects the fifth element of a Polylist.	181
Object	<u>first</u> () first() returns the first element of a non-empty Polylist.	174
static Object	<u>first</u> (Polylist L) first(L) returns the first element of its argument.	180
Object	<u>fourth</u> () fourth selects the fourth element of a Polylist.	181
String	<u>implode</u> () implode() creates a String from a Polylist of items	183
String	<u>implode</u> (String separator)	184
boolean	<u>isEmpty</u> () isEmpty() tells whether the Polylist is empty.	174
int	<u>length</u> () return the length of this list	180
static Polylist	<u>list</u> () return a list of no elements	175
static Polylist	<u>list</u> (Object A) return a list of one element	176
static Polylist	<u>list</u> (Object A, Object B) return a list of two elements	176
static Polylist	<u>list</u> (Object A, Object B, Object C) return a list of three elements	176
static Polylist	<u>list</u> (Object A, Object B, Object C, Object D) return a list of four elements	176
static Polylist	<u>list</u> (Object A, Object B, Object C, Object D, Object E) return a list of five elements	176
static Polylist	<u>list</u> (Object A, Object B, Object C, Object D, Object E, Object F) return a list of six elements	176
static Polylist	<u>list</u> (Object A, Object B, Object C, Object D, Object E, Object F, Object G) return a list of seven elements	177

static Polylist	<u>list</u> (Object A, Object B, Object C, Object D, Object E, Object F, Object G, Object H) return a list of eight elements	177
static Polylist	<u>list</u> (Object A, Object B, Object C, Object D, Object E, Object F, Object G, Object H, Object I) return a list of nine elements	177
static Polylist	<u>list</u> (Object A, Object B, Object C, Object D, Object E, Object F, Object G, Object H, Object I, Object J) return a list of ten elements	177
static Polylist	<u>list</u> (Object A, Object B, Object C, Object D, Object E, Object F, Object G, Object H, Object I, Object J, Object K) return a list of eleven elements	178
static Polylist	<u>list</u> (Object A, Object B, Object C, Object D, Object E, Object F, Object G, Object H, Object I, Object J, Object K, Object L) return a list of twelve elements	178
static Polylist	<u>list</u> (Object A, Object B, Object C, Object D, Object E, Object F, Object G, Object H, Object I, Object J, Object K, Object L, Object M) return a list of thirteen elements	178
static Polylist	<u>list</u> (Object A, Object B, Object C, Object D, Object E, Object F, Object G, Object H, Object I, Object J, Object K, Object L, Object M, Object N) return a list of fourteen elements	179
static Polylist	<u>list</u> (Object A, Object B, Object C, Object D, Object E, Object F, Object G, Object H, Object I, Object J, Object K, Object L, Object M, Object N, Object O) return a list of fifteen elements	179
static Polylist	<u>list</u> (Object A, Object B, Object C, Object D, Object E, Object F, Object G, Object H, Object I, Object J, Object K, Object L, Object M, Object N, Object O, Object P) return a list of sixteen elements	179
static void	<u>main</u> (String[] args) test program for Polylists etc.	183
Polylist	<u>map</u> (Function1 F) map maps an object of class Function1 over a Polylist returning a Polylist	184
boolean	<u>member</u> (Object A) member(A, L) tells whether A is a member of this	180
boolean	<u>nonEmpty</u> () nonEmpty() tells whether the Polylist is non-empty.	174
Object	<u>nth</u> (long n) nth selects Polylist item by index (0, 1, 2, ...).	182
static Polylist	<u>PolylistFromArray</u> (Object[] array) PolylistFromArray makes a list out of an array of objects	183
static Polylist	<u>PolylistFromEnum</u> (Enumeration e) PolylistFromEnum makes a Polylist out of any Enumeration.	175
Polylist	<u>prefix</u> (long n) prefix creates the length-n prefix of a Polylist.	182
static Polylist	<u>range</u> (long M, long N) range(M, N) returns a Polylist of the form (M M+1	181
static Polylist	<u>range</u> (long M, long N, long S) range(M, N, S) returns a Polylist of the form (M M+S	181

Object	<u>reduce</u> (Function2 F, Object unit) reduce reduces a Polylist by a Function2 object, with unit	184
Polylist	<u>rest</u> () rest() returns the rest of a non-empty Polylist.	174
static Polylist	<u>rest</u> (Polylist L) rest(L) returns the rest of its argument.	180
Polylist	<u>reverse</u> () reverse(L) returns the reverse of this	180
Object	<u>second</u> () second selects the second element of a Polylist.	181
void	<u>setFirst</u> (Object ob) setFirst() sets the first of a list to an object	174
void	<u>setNth</u> (long n, Object o)	182
Object	<u>sixth</u> () sixth selects the sixth element of a Polylist.	182
Object	<u>third</u> () third selects the third element of a Polylist.	181
String	<u>toString</u> () toString() converts Polylist to string, e.g.	175
String	<u>toStringSansParens</u> () toString() converts Polylist to string, e.g.	175

Field Detail

nil

```
public static final Polylist nil
```

nil is the empty-list constant

Constructor Detail

Polylist

```
public Polylist()
```

construct empty Polylist

Polylist

```
public Polylist(Object First,  
                Polylist Rest)
```

construct non-empty Polylist from a First and Rest

Polylist

```
public Polylist(Object First,  
                polyA.Seed Rest)
```

construct non-empty Polylist from a First and a Seed for the Rest

Method Detail

isEmpty

```
public boolean isEmpty()
```

isEmpty() tells whether the Polylist is empty.

nonEmpty

```
public boolean nonEmpty()
```

nonEmpty() tells whether the Polylist is non-empty.

first

```
public Object first()
```

first() returns the first element of a non-empty Polylist.

Throws:

NullPointerException - Can't take first of an empty Polylist.

setFirst

```
public void setFirst(Object ob)
```

setFirst() sets the first of a list to an object

Throws:

NullPointerException - Can't take first of an empty Polylist.

rest

```
public Polylist rest()
```

rest() returns the rest of a non-empty Polylist.

Throws:

NullPointerException - Can't take rest of an empty Polylist.

toString

```
public String toString()
```

toString() converts Polylist to string, e.g. for printing

Overrides:

toString in class *Object*

toStringSansParens

```
public String toStringSansParens()
```

toString() converts Polylist to string, e.g. for printing

cons

```
public Polylist cons(Object First)
```

cons returns a new Polylist given a First and this as a Rest

cons

```
public static Polylist cons(Object First,  
                             Polylist Rest)
```

static cons returns a new Polylist given a First and a Rest

cons

```
public static Polylist cons(Object First,  
                             polya.Seed Rest)
```

This variant of cons takes a Seed instead of a Polylist as rest, so the list can be grown incrementally.

PolylistFromEnum

```
public static Polylist PolylistFromEnum(Enumeration e)
```

PolylistFromEnum makes a Polylist out of any Enumeration.

list

```
public static Polylist list()
```

return a list of no elements

list

```
public static Polylist list(Object A)
```

return a list of one element

list

```
public static Polylist list(Object A,  
                             Object B)
```

return a list of two elements

list

```
public static Polylist list(Object A,  
                             Object B,  
                             Object C)
```

return a list of three elements

list

```
public static Polylist list(Object A,  
                             Object B,  
                             Object C,  
                             Object D)
```

return a list of four elements

list

```
public static Polylist list(Object A,  
                             Object B,  
                             Object C,  
                             Object D,  
                             Object E)
```

return a list of five elements

list

```
public static Polylist list(Object A,  
                             Object B,  
                             Object C,  
                             Object D,  
                             Object E,  
                             Object F)
```

return a list of six elements

list

```
public static Polylist list(Object A,  
                             Object B,  
                             Object C,  
                             Object D,  
                             Object E,  
                             Object F,  
                             Object G)
```

return a list of seven elements

list

```
public static Polylist list(Object A,  
                             Object B,  
                             Object C,  
                             Object D,  
                             Object E,  
                             Object F,  
                             Object G,  
                             Object H)
```

return a list of eight elements

list

```
public static Polylist list(Object A,  
                             Object B,  
                             Object C,  
                             Object D,  
                             Object E,  
                             Object F,  
                             Object G,  
                             Object H,  
                             Object I)
```

return a list of nine elements

list

```
public static Polylist list(Object A,  
                             Object B,  
                             Object C,  
                             Object D,  
                             Object E,  
                             Object F,  
                             Object G,  
                             Object H,  
                             Object I,  
                             Object J)
```

return a list of ten elements

list

```
public static Polylist list(Object A,  
                             Object B,  
                             Object C,  
                             Object D,  
                             Object E,  
                             Object F,  
                             Object G,  
                             Object H,  
                             Object I,  
                             Object J,  
                             Object K)
```

return a list of eleven elements

list

```
public static Polylist list(Object A,  
                             Object B,  
                             Object C,  
                             Object D,  
                             Object E,  
                             Object F,  
                             Object G,  
                             Object H,  
                             Object I,  
                             Object J,  
                             Object K,  
                             Object L)
```

return a list of twelve elements

list

```
public static Polylist list(Object A,  
                             Object B,  
                             Object C,  
                             Object D,  
                             Object E,  
                             Object F,  
                             Object G,  
                             Object H,  
                             Object I,  
                             Object J,  
                             Object K,  
                             Object L,  
                             Object M)
```

return a list of thirteen elements

list

```
public static Polylist list(Object A,  
                             Object B,  
                             Object C,  
                             Object D,  
                             Object E,  
                             Object F,  
                             Object G,  
                             Object H,  
                             Object I,  
                             Object J,  
                             Object K,  
                             Object L,  
                             Object M,  
                             Object N)
```

return a list of fourteen elements

list

```
public static Polylist list(Object A,  
                             Object B,  
                             Object C,  
                             Object D,  
                             Object E,  
                             Object F,  
                             Object G,  
                             Object H,  
                             Object I,  
                             Object J,  
                             Object K,  
                             Object L,  
                             Object M,  
                             Object N,  
                             Object O)
```

return a list of fifteen elements

list

```
public static Polylist list(Object A,  
                             Object B,  
                             Object C,  
                             Object D,  
                             Object E,  
                             Object F,  
                             Object G,  
                             Object H,  
                             Object I,  
                             Object J,  
                             Object K,  
                             Object L,  
                             Object M,  
                             Object N,  
                             Object O,  
                             Object P)
```

return a list of sixteen elements

length

```
public int length()
```

return the length of this list

elements

```
public PolylistEnum elements()
```

elements() returns a PolylistEnum object, which implements the interface java.util.Enumeration.

first

```
public static Object first(Polylist L)
```

first(L) returns the first element of its argument.

Throws:

NullPointerException - Can't take first of empty List.

rest

```
public static Polylist rest(Polylist L)
```

rest(L) returns the rest of its argument.

Throws:

NullPointerException - Can't take rest of empty Polylist.

reverse

```
public Polylist reverse()
```

reverse(L) returns the reverse of this

append

```
public Polylist append(Polylist M)
```

append(M) returns a Polylist consisting of the elements of this followed by those of M.

member

```
public boolean member(Object A)
```

member(A, L) tells whether A is a member of this

range

```
public static Polylist range(long M,  
                             long N)
```

range(M, N) returns a Polylist of the form (M M+1 N)

range

```
public static Polylist range(long M,  
                             long N,  
                             long S)
```

range(M, N, S) returns a Polylist of the form (M M+S N)

second

```
public Object second()
```

second selects the second element of a Polylist.

Throws:

NullPointerException - Can't take second of Polylist.

third

```
public Object third()
```

third selects the third element of a Polylist.

Throws:

NullPointerException - Can't take third of Polylist.

fourth

```
public Object fourth()
```

fourth selects the fourth element of a Polylist.

Throws:

NullPointerException - Can't take fourth of Polylist.

fifth

```
public Object fifth()
```

fifth selects the fifth element of a Polylist.

Throws:

NullPointerException - Can't take fifth of Polylist

sixth

public Object **sixth**()

sixth selects the sixth element of a Polylist.

Throws:

NullPointerException - Can't take sixth of Polylist

nth

public Object **nth**(long n)

nth selects Polylist item by index (0, 1, 2, ...).

Throws:

NullPointerException - Can't select from an empty Polylist.

setNth

public void **setNth**(long n,
Object o)

prefix

public Polylist **prefix**(long n)

prefix creates the length-n prefix of a Polylist.

coprefix

public Polylist **coprefix**(long n)

coprefix creates the Polylist with all but the length-n prefix of a Polylist

equals

public static boolean **equals**(Polylist L,
Polylist M)

equals(L, M) tells whether Polylists L and M are equal

equals

```
public boolean equals(Object M)
```

equals(M) tells whether this Polylist is equal to some other Object

Overrides:

equals in class Object

main

```
public static void main(String[] args)
```

test program for Polylists etc.

analysis

```
public static String analysis(Object Ob)
```

analysis produces a string analyzing objects, especially Polylists

array

```
public Object[] array()
```

array() returns an array of elements in list

PolylistFromArray

```
public static Polylist PolylistFromArray(Object[] array)
```

PolylistFromArray makes a list out of an array of objects

explode

```
public static Polylist explode(String S)
```

explode(String S) converts a string into a Polylist of Character

implode

```
public String implode()
```

implode() creates a String from a Polylist of items

implode

```
public String implode(String separator)
```

map

```
public Polylist map(Function1 F)
```

map maps an object of class Function1 over a Polylist returning a Polylist

reduce

```
public Object reduce(Function2 F,  
                    Object unit)
```

reduce reduces a Polylist by a Function2 object, with unit

assoc

```
public static Polylist assoc(Object ob,  
                             Polylist alist)
```

assoc

```
public Polylist assoc(Object ob)
```


Class PolylistEnum

polya

```
java.lang.Object
└─ polya.PolylistEnum
```

All Implemented Interfaces:
Enumeration

```
public class PolylistEnum
extends Object
implements Enumeration
```

PolylistEnum is an enumeration class for the class Polylist. It implements the interface `java.util.Enumeration`, i.e. the methods: `hasMoreElements()` and `nextElement()`.

Author:
Robert Keller

Constructor Summary	Page
<u>PolylistEnum</u> (Polylist L) PolylistEnum constructs a PolylistEnum from a Polylist.	185

Method Summary	Page
boolean <u>hasMoreElements</u> () hasMoreElements() indicates whether there are more elements left in the enumeration.	185
Object <u>nextElement</u> () nextElement returns the next element in the enumeration.	186

Constructor Detail

PolylistEnum

```
public PolylistEnum(Polylist L)
```

PolylistEnum constructs a PolylistEnum from a Polylist.

Method Detail

hasMoreElements

```
public boolean hasMoreElements()
```

hasMoreElements() indicates whether there are more elements left in the enumeration.

Specified by:
hasMoreElements in interface Enumeration

nextElement

`public Object nextElement()`

nextElement returns the next element in the enumeration.

Specified by:

nextElement in interface *Enumeration*

Class Tokenizer

polya

```
java.lang.Object
├─ java.io.StreamTokenizer
│   └─ polya.Tokenizer
```

```
public class Tokenizer
extends StreamTokenizer
```

Tokenizer provides a tokenizer of the package *polya*.

In addition to tokenizing for values of specific types, an entire Poly can be read by one method call, assuming the input is in the form of an S expression or an R expression.

In an S expression, the elements of a Polylist are shown in parentheses with a space between each element. (The elements can themselves be Polylists.)

In an R expression, the elements are shown in square brackets, with an optional comma separator between them.

Author:

Robert Keller

Field Summary		Page
static EOF	<u>eof</u>	188
long	<u>lval</u>	188
static int	<u>TT LONG</u>	188

Constructor Summary		Page
<u>Tokenizer</u> (InputStream I) construct tokenizer from input stream		188
<u>Tokenizer</u> (Reader R) construct tokenizer from Reader		188

Method Summary		Page
void	<u>exceptionHandler</u> (Exception e) exceptionHandler can be over-ridden to handle IO exceptions	188
void	<u>getSkippingWhiteSpace</u> ()	188
static void	<u>main</u> (String[] args) Test method: loops through input, printing out each type of token.	189
Object	<u>nextRexp</u> () get next R expression from input stream	189
Object	<u>nextSexp</u> () get next S expression from input stream returns an object of class EOF on end-of-file	189

int	nextToken() get token, indicating TT_LONG, TT_NUMBER, TT_WORD, or TT_EOF	188
-----	--	-----

Field Detail

eof

```
public static final EOF eof
```

lval

```
public long lval
```

TT_LONG

```
public static final int TT_LONG
```

Constructor Detail

Tokenizer

```
public Tokenizer(InputStream I)

    construct tokenizer from input stream
```

Tokenizer

```
public Tokenizer(Reader R)

    construct tokenizer from Reader
```

Method Detail

exceptionHandler

```
public void exceptionHandler(Exception e)

    exceptionHandler can be over-ridden to handle IO exceptions
```

getSkippingWhiteSpace

```
public void getSkippingWhiteSpace()
```

nextToken

```
public int nextToken()
```


get token, indicating TT_LONG, TT_NUMBER, TT_WORD, or TT_EOF

Overrides:

nextToken in class StreamTokenizer

nextSexp

public Object nextSexp()

get next S expression from input stream returns an object of class EOF on end-of-file

nextRexp

public Object nextRexp()

get next R expression from input stream

main

public static void main(String[] args)

Test method: loops through input, printing out each type of token. Terminates on end-of-file, with test of S expressions, then R expressions.